

Diretrizes para Desenvolvimento: Simulador de Teoria da Computação (SIN 141)

Este documento atua como especificação técnica detalhada para orientar o desenvolvimento de um sistema robusto de manipulação de Autômatos Finitos e Gramáticas Regulares em **Java**.

1. Visão Geral e Requisitos de Escopo

O objetivo do sistema é implementar, de forma puramente algorítmica (sem o uso de bibliotecas de processamento regex externas), as estruturas de dados e operações fundamentais de linguagens regulares da Teoria da Computação.

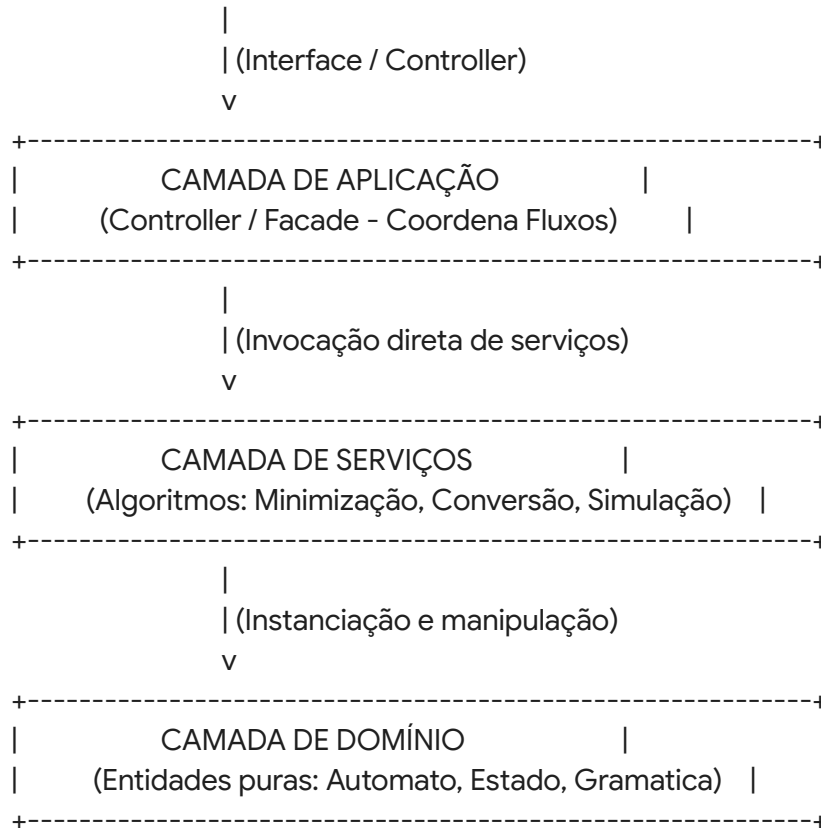
Requisitos Funcionais do Core Algorítmico

1. **Modelagem de AFN:** Criação e representação de Autômatos Finitos Não-Determinísticos (com suporte a transições em vazio ϵ).
2. **Conversão AFN $\rightarrow$ AFD:** Algoritmo de Construção de Subconjuntos para conversão exata de AFN para AFD equivalente.
3. **Minimização de AFD:** Implementação de algoritmo clássico de particionamento (ex: Algoritmo de Hopcroft ou Moore) para obter o AFD mínimo correspondente.
4. **Simulação de Palavra:** Algoritmo de validação/reconhecimento de cadeias de símbolos através de um AFD já determinizado.
5. **Conversão de GR $\rightarrow$ AF:** Mapeamento de produções de uma Gramática Regular para transições de um autômato finito equivalente.
6. **Conversão de AF $\rightarrow$ GR:** Extração da Gramática Regular equivalente a partir de um Autômato Finito fornecido.

2. Decisões Arquiteturais e Desacoplamento

Para assegurar uma base de código flexível, modular e escalável, o projeto deverá seguir uma variação simplificada de **Arquitetura em Camadas (Clean Architecture / MVC)**. A lógica de negócio (representação matemática e algoritmos) deve ser completamente agnóstica de frameworks de interface gráfica.

```
+-----+
|          CAMADA DE APRESENTAÇÃO          |
| (Java Swing ou JavaFX - Telas, Inputs, Desenho) |
+-----+
```



3. Planejamento das Classes e Domínio (Java)

O agente deve gerar as classes organizadas em pacotes estruturados. Abaixo está a definição sugerida de pacotes e suas responsabilidades:

br.ufv.sin141.domain.model (Domínio Puro)

Entidades matemáticas imutáveis ou puras de estado.

- State: Classe que representa um estado do autômato.
 - *Campos*: String id, boolean isInitial, boolean isFinal.
- Transition: Classe genérica ou específica de mapeamento.
 - *Campos*: State origin, String symbol, Set<State> targets. (No caso de AFD, o conjunto de alvos terá tamanho no máximo 1).
- Automaton: Classe abstrata ou representação unificada de um autômato.
 - *Campos*: Set<State> states, Set<String> alphabet, Set<Transition> transitions, State initialState.
 - *Métodos úteis*: getTransitionsFrom(State s, String symbol).
- ProductionRule: Representação de regra de produção gramatical ($A \rightarrow aB$ ou $A \rightarrow a$).

- *Campos*: String leftHandSide, String rightHandSideTerminal, String rightHandSideNonTerminal (com suporte a vazio).
- RegularGrammar: Representação formal de uma Gramática Regular.
 - *Campos*: Set<String> nonTerminals, Set<String> terminals, Set<ProductionRule> rules, String startSymbol.

br.ufv.sin141.domain.service (Algoritmos e Lógica)

Processamento pesado e transformações matemáticas. **Essas classes recebem estruturas de domínio e retornam novas estruturas de domínio modificadas.**

- SubsetConstructionService:
 - Dfa convertNfaToDfa(Nfa nfa): Implementa o fechamento- ϵ e o mapeamento de subconjuntos.
- HopcroftMinimizationService:
 - Dfa minimizeDfa(Dfa dfa): Executa a partição clássica de estados equivalentes e distinguíveis.
- GrammarConverterService:
 - Automaton toAutomaton(RegularGrammar grammar)
 - RegularGrammar toGrammar(Automaton automaton)
- SimulationService:
 - boolean simulate(Dfa dfa, String word): Realiza a transição de estados determinística para avaliar a aceitação.

br.ufv.sin141.application.controller (Orquestração)

Controlador único que atua como porta de entrada (Facade) para as telas do sistema. Ele impede o acoplamento direto entre os cliques de botões na UI e a implementação do algoritmo.

- TheoryAppController:
 - *Campos*: Instâncias dos serviços descritos acima.
 - *Responsabilidade*: Expor métodos que retornam representações fáceis para a View renderizar (ex: strings de transição formatadas, dados para preencher tabelas, etc.).

4. Camada de Interface Gráfica (GUI)

Toda a captura de dados de entrada de autômatos ou gramáticas deve ser feita por meio de uma interface rica, fornecendo feedback visual amigável dos resultados computados.

Tecnologias Recomendadas para a UI

- **JavaFX** (Recomendado para melhor apelo visual, organização por CSS e manutenibilidade usando padrões FXML/Controller de tela).
- **Alternativa: Java Swing** (Usando layouts puros e estruturando painéis estáticos bem organizados para garantir portabilidade direta sem dependências adicionais).

Telas e Fluxos de Interface

1. Painel de Definição de Autômato (AFN/AFD):

- Interface de tabela ou lista para cadastro dinâmico de estados e definição se são iniciais/finais.
- Definição interativa do alfabeto.
- Grade de transições em matriz ou lista dinâmica de conexões.

2. Painel de Conversões:

- Painel onde o usuário visualiza o autômato original.
- Botão "Converter AFN para AFD". O resultado deve ser exibido em uma segunda tabela/painel gráfico paralelo.
- Botão "Minimizar AFD", aplicando o algoritmo de redução e exibindo a nova tabela de transições otimizada.

3. Painel de Simulação:

- Input de texto contínuo para inserção de uma palavra.
- Botão "Simular Entrada".
- **Feedback Visual:** Painel exibindo "ACEITA" (cor verde) ou "REJEITADA" (cor vermelha), de preferência exibindo a sequência/caminho percorrido de estados até o resultado.

4. Painel de Gramática Regular:

- Campo para inserção amigável de regras gramaticais (ex: $S \rightarrow aA \mid bB \mid e$).
- Botão de conversão mútua "Gramática $\rightarrow$ Autômato" e "Autômato $\rightarrow$ Gramática".

5. Diretrizes de Robustez e Tratamento de Erros

A aplicação não deve falhar (crashar) em hipótese alguma devido a dados corrompidos. A camada de aplicação deve conter tratamento completo de exceções:

- **AlphabetViolationException:** Lançada ao tentar simular uma palavra que contém caracteres fora do alfabeto definido para o autômato.
- **MalformedGrammarException:** Lançada quando o usuário insere produções inválidas (que fogem do escopo de Gramáticas Regulares).
- **NondeterministicTransitionException:** Lançada caso o motor de simulação detecte indeterminações ao tentar processar palavras sem a devida determinização prévia.

6. Instruções Específicas para Geração de Código do Gemini

Ao codificar este projeto, garanta que:

1. **Criação de Pacotes:** Os arquivos correspondam ao layout clássico de projetos Java baseado em `src/main/java`.
2. **Boas Práticas:** Use tipagem forte no Java (`Set<State>`, `Map<Pair<State, String>`, `Set<State>>`, etc.). Evite strings puras para gerenciar estados internos.
3. **Visualização Gráfica Decoupled:** Use os painéis gráficos apenas para exibir os dados contidos nas entidades (`Automaton`, `RegularGrammar`). A view deve pedir dados

formatados ao TheoryAppController ao invés de calcular as transições localmente.

eof